DuckDB and the changing physics of analytics

August 26, 2026 • 3554 words

For as long as most of us have been building with data, the systems we reach for — databases, query engines, data warehouses — have, at any appreciable scale, been separate systems. As a community, we've generated a lot of healthy arguments about their design along the way. Single host, clustered, or distributed, whether data should all live in memory, whether throughput or latency was the thing that mattered most, but almost all of them have been big systems that live on the other side of a wire. And that's changing, because the relative costs of compute, memory, and network on a single machine are not the constraints they once were, and a lot of the work we used to send away no longer needs to leave the application.

In this post, Andy Warfield explains how databases like DuckDB are enabling a new way to build with data, why they matter right now, and how they complement the work we've been doing in S3 (e.g., S3 Files, S3 Tables, S3 Vectors). And most importantly, why DuckLabs, the team behind DuckDB, is joining AWS.

Enjoy.

—W

A (somewhat uncomfortably) long time ago in a British pub

Possibly even more than other places, evenings at the pub are an integral part of grad school in the UK. I still remember in those conversations that some of my math and physics friends would poke fun at the computer scientists: “any discipline that needs to put science in its name probably isn’t a science.” I’m sure that I said some uncomfortable things back to them, but there was a grain of truth to what they were saying. It was a weird grain of truth though because it was grating enough to stick with me, and the more I thought about it, the more I realized that by their definition, ***not being a science*** was possibly the thing that I loved the most about working on computer systems.

Here’s what I mean: where the physical sciences explore the world as it is and look for deeper and deeper understandings of an immovable, inevitable truth, computer science — and especially computer systems — has always been about finding the most elegant solution to a problem in the face of changing “physics”. The relative differences between things, like the speed of memory compared to the speed of the network, the richness of software abstractions relative to available compute power, and so on. It’s an art trying to engineer elegant compromises, to do your best in a world of invariants that don’t stay still over time. And that even the success of your own system can wind up pushing you into a different scale of operation, and a different set of constraints than the ones you started out with.

There are loads of really neat examples of these tradeoffs shaping systems design. The Berkeley Network of Workstations (NOW) project was one of the first systems research projects that I remember really capturing my imagination as a computer science student. NOW capitalized on relatively inexpensive compute and the emerging bandwidth of commodity networks to build interesting scalable distributed systems. Later, the graduate work I was involved in with the Xen hypervisor took advantage of the opposite trend: Xen used the increasing abundance of CPU, memory, and network capabilities on individual servers to build isolated virtual machines and allowed the same hardware to be used by mutually untrusting tenants. Database design has always been about making the most of the current hardware capabilities. To pick a single example, at CWI in Amsterdam, the MonetDB and X100 work took advantage of the fact that the bottleneck in query processing had quietly moved off the disk and into the CPU — cache behaviour, branch prediction, superscalar pipelines — to rebuild query execution around batches of values small enough to stay in cache. All of these systems are interesting because of how they reimagined design in the face of changing constraints and evolving physics.

A fun bit of this is that these types of constraints tend to run in cycles. A lot of the virtualization ideas that we explored in Xen were established in the 60s on IBM mainframes. Despite wildly different form factors between massive mainframes and commodity servers, the common theme at those two moments in time was that CPU was abundant and finer-grained isolation was desirable. This cyclical nature of systems design also means that as physics evolve and new designs emerge there's often an initial reaction along the lines of "we did all this in the 70s," but then a bit of a more nuanced realization that we are coming back to old ideas with fresh eyes and the benefit of other lessons we've learned since the last time.

One place where the physics have changed an awful lot and have wildly influenced system design is in data processing. Processing data, especially for reasonably sophisticated queries, turns into a very complicated systems problem very quickly. There's a direct tension between the complexity of the query and the scale of the data being processed, which is that data dependencies between data in different parts of a data set make parallelization difficult and inefficient because you need more and more fast memory to work across those dependent bits of data, and parallelization, either because of distribution or concurrency control, inevitably means doing stuff that complicates and slows down that memory. For this reason, processing tasks are always easiest and most efficient to deal with on a single, fast computer, BUT, the volume of data that we deal with just keeps growing and growing. And so, when we need to process really big data sets and the constrained resource (the physics of the day) is our ability to read it fast enough – where the NIC or the disk on a single server just can't even make it through the volume of data we want to process in, say, a single day — we tend to reach for distributed processing tools. We find ways to partition the processing task, we spread it out across a lot of parallel servers, we do as much as we can in partitioned parallel execution, and then we combine the results (hopefully as a much more compact set of intermediate data with a lot less work required) into a single answer.

Limited I/O bandwidth to large datasets was certainly the physics of the day in the early 2000s, and so scale-out processing was the core idea behind both Google's work on MapReduce, and later behind Spark's Resilient Distributed Datasets (RDDs). These systems were all conceived in the face of two invariants (from that time): the data sets that we wanted to process were getting pretty darned big, and the single host commodity network interface was comparatively small. These systems solved a really important problem in working with data and have had enormous influence on how we do data processing at almost every scale today.

I've always found two aspects of these distributed query processing systems to be remarkable. First, they innovated on the developer ergonomics of data processing. I think we take this a little bit for granted sometimes, but MapReduce, as an example, observed that a pretty simple pattern from functional programming (database people will pedantically point out that it should actually be called Map/Group-by-

and-aggregate, but it's a little less catchy) could sneakily force developers to articulate data processing tasks in a way that made the opportunity for parallelism very, very explicit. Spark took this even further, extending the idea of lazily evaluated operator chaining (and eventually dataframes) into something that was relatively easy to program, but could then be passed behind the scenes to a query planner and scheduler in order to decompose the developer's analysis code into a collection of tasks that could be dispatched onto a distributed set of computers. These systems and interfaces did a really good job at helping developers ignore the fact that their code was running on the other side of a wire. They took the physical necessity of having a remote, distributed system, and helped the developer work through the constraints of that system without having to think about splitting up their query, sending stuff over the network, dealing with failures, and so on.

The second thing I find fascinating is the way these systems tend to think about performance. The tradeoff that they've often made, and it's a very reasonable one, is to engineer for arbitrarily high parallelism and throughput. They are comfortable paying an up-front cost for planning jobs, for shipping tasks around, and the other overheads of distribution because the throughput benefit of parallelism was so meaningful. The ability to drive throughput, and the availability of enormous amounts of parallel compute in cloud environments, has meant that these systems have tended to achieve that throughput by adding additional computers to the processing job rather than by being lean.

I mean this much more as an observation than as a criticism of these systems, because they were building for their own physics. The fact that they were distributed systems meant that queries would always be sent to run remotely – we were always talking to a server at the other end of a wire, and so we built ergonomic developer abstractions that made that manageable and set the appropriate expectations. If we needed more throughput or to deal with larger data sets, we could just run jobs on even bigger clusters. But it's interesting to think about how this necessarily remote structure created its own physics for analytics developers: developers learned to expect latency in running jobs and they grew to depend on having a remote cluster to be able to do meaningful processing.

> "You can have a second computer once you've shown you know how to use the first one." –Paul Barham

While all of this distributed query processing work has been evolving, something has been happening to the physics of the systems that motivated them. Nineteen years ago, a then-new m1.xlarge (the beefiest EC2 instance on AWS at the time) had 15 GB of RAM, 4 virtual cores, and roughly 1 Gb/s of network connectivity. Today, a single m8g.48xlarge has about 50× more memory, 50× more cores, and 50× more network bandwidth. The servers that we work on today have more parallelism and aggregate I/O bandwidth than the clusters that many of us first ran

Hadoop and Spark on. Heck, even the MacBook Pro that I'm writing on right now has 3-5x the CPU cores and RAM, about 40x the memory bandwidth and over 100x the I/O bandwidth of that m1.xlarge. And yes, the data sets have grown too, but the thing about data set growth is that it happens along a distribution. The largest data sets have become exponentially larger, but those are the tail. Many data sets scale with very human things: the size of a business, the number of customers it serves, or the number of bank transactions a person makes in a day. This growing hardware advantage relative to data size has driven a resurgence of interest in extremely efficient single-host engines.

In 2015, Frank McSherry, Michael Isard, and Derek Murray wrote a wonderfully irreverent paper, "Scalability! But at what COST?," about exactly this topic, poking a bit of fun at the (in)efficiency of large distributed processing systems. They took some common data processing tasks, built a good single-threaded version, and compared it to the scale a distributed framework needed to match that performance. It was an enjoyable and surprising read because they showed, for example, that a well-optimized implementation running on a single thread could beat distributed graph systems running on 128 cores, and that it took 512 cores before the distributed version finally pulled ahead. The paper was punctuated by the fact that the authors all worked on distributed data processing systems themselves, and so it wasn't so much a poke at distributed designs as it was a call to pay attention to per-core efficiency.

Their work resonated with (and I think continues to influence) performance-leaning systems people, and I was reminded of it in one of my first conversations with Hannes Mühleisen and Mark Raasveldt, the creators of DuckDB — when over pints of beer in Amsterdam, they quoted Paul Barham's epigraph in the COST paper: "You can have a second computer once you've shown you know how to use the first one."

If it walks like a duck…

I'm not saying the COST paper was the specific motivation for DuckDB, but it captured a sentiment that Hannes and Mark clearly agreed with. They'd both been researchers at CWI, in the same lab that produced the MonetDB and X100 work I mentioned earlier, so they came at analytics with a deep interest in ideas like vectorization and the goal of getting everything they could out of a single CPU. When they started the project in 2018, that focus on efficiency led them somewhere different from distributed systems, toward an analytics engine that runs as a library, in the same address space as the application that wants to process data. Their 2019 SIGMOD demo paper used the popularity of SQLite as an example of how effective it could be to package an engine to be embedded inside applications and argued for the potential value of an efficient analytics-focused data processing library.

Running as an embedded library was a clever reimagining of where the “work” happens, working in-situ on the same in-memory data structures you already use for your tables, and caring as much about its own overhead as about the queries it runs. The research project led to a wildly popular open-source engine, called DuckDB. (Fun aside: “Duck” was because the team felt that database projects tended to use a lot of names that emphasized hyper-awesome-ultra-performance and they found it grating. Shortly before starting the project, Hannes had a pet duck named Wilbur, possibly the most influential duck in the history of computer science.) Importantly, the fact that it’s structured as an embedded library doesn’t mean that it has to always be in the client application. The library form factor just means that it doesn’t have to exist as an external service, usually on the other end of the wire. Instead, the library turns into a building block that can be placed at the appropriate point(s) in the software stack to be effective. The most extreme version of that is that the engine compiles to WebAssembly and runs entirely inside a browser tab, which you can see for yourself at https://shell.duckdb.org/.

The S3 team started to see exactly this sort of application-driven use in our own customer workloads. Application builders were embedding DuckDB directly to work with their data, and that pattern has only accelerated since. It’s what led me to reach out to the DuckLabs team while we were working on S3 Tables. We had decided to extend S3 with a first-class tabular storage primitive built on Iceberg, because some of our largest Spark customers were adopting Iceberg and wanted storage that was an intentional fit for tabular data. Having watched this excitement around DuckDB in a different set of our customers, I wanted to be sure S3 Tables would be a great fit for the DuckDB users as well. And that’s how I found myself sitting with Hannes and Mark, over those pints in Amsterdam.

…and quacks like a duck

We spent a bunch of time talking about systems in those early conversations and I think we found that across S3 and DuckDB, we shared a lot of the same views toward building software, enabling developers, and working with data. Hannes has described their goal as allowing “anyone to work with data confidently”, a framing I absolutely love, because of the emphasis Hannes puts on the “confidently” part when he says it. We quickly agreed that AWS would become their customer and sponsor their Iceberg extension, with a goal of broadening support for Iceberg outside of the Spark landscape and making it as simple as possible to build applications over S3 Tables. Since then, that extension has grown to be a very mature implementation of both the Iceberg v2 and v3 specifications, and has motivated the async I/O support that is about to arrive in the 2.0 release. In building async I/O, their team took the goal of being able to saturate the NIC while scanning tables from S3 – it’s a really nice bit of performance work. And the extension has

proven to be pretty popular with the DuckDB community, receiving over 800K downloads a week.

Over the past two years (give or take), our teams have worked more and more closely together and have broadened the areas where we've collaborated. We explored AWS Lambda as a matched primitive for launching DuckDB queries quickly, and we smoothed the integrations between DuckDB and the AWS database and analytics engines through DuckDB's evolving ATTACH and CONNECT commands.

The DuckDB community has taught us that when the engine is just a library, the line between building an application and analyzing its data gets a lot blurrier. You reach for the same engine when you're prototyping against a pile of CSV files, when you add a feature to your application that needs to aggregate something, when you build a second application against the same data, and when you eventually want to report on all of it. It's the same SQL and the same engine at every step, which means you aren't making a decision on day one about what your data is eventually going to become.

None of which displaces the need for the distributed approaches we already have. When a job genuinely needs a thousand machines, it needs a thousand machines, and approaches that made that tractable aren't going anywhere. What's changed is that an enormous amount of the data work people actually do never needed a cluster in the first place, and now it doesn't have one.

The library form factor and focus on portability also means that a single, efficient code base can live at many places in the stack and result in improvements "raising all ships." Somewhere along the way I started describing DuckDB to people as "the glibc of structured data": a lean, unglamorous, ubiquitous dependency that a great deal of software links against and almost nobody has to think about. DuckDB's extensibility and its ability to be used across so many data formats, existing database interfaces, and in application form factors ranging from browsers to server-side engine code make it applicable to a huge range of problems. The shift I find most interesting isn't really about where the engine runs — it's that "analytics" is becoming less of a separate activity that happens to data somewhere else, and more of something you do continuously as you build.

…it must be a duck

Hannes and Mark's vision is so aligned with the way that we think about data at Amazon that we wanted to give them the broadest possible opportunity to set the agenda not just for DuckDB, but for how analytics should be built and delivered in the cloud. So, after many conversations (not all in pubs, I swear), we jointly made

the decision to have the DuckLabs team join AWS (and we start working together this week).

From an AWS perspective, we want to continue to evolve DuckDB to be the most natural tool that developers, and increasingly agents, reach for when they work with structured data. We're already using it internally for dashboards, CLI tooling, in-server accelerators, and bridges between systems, and many of our customers use it even more broadly than that. We want to support the project's evolution in a way that preserves exactly what makes it great: the incredible community, the remarkable development velocity that the team has maintained over the past eight years, and the broad and growing set of application domains where DuckDB is effective.

DuckLabs is joining AWS as a subsidiary, and the DuckDB project will continue to operate as an open source project under the stewardship of the DuckDB Foundation, developed by the DuckLabs team, and be available under an MIT license. Hannes and Mark have talked about this evolution in their own words in a blog post on the DuckLabs blog. The DuckLabs team will continue to operate as they do today from their current office in Amsterdam, where Hannes's architect brother has designed much of the office's decor. As part of the broader AWS team, we'll all work to accelerate DuckDB's already incredible velocity.

The SIGMOD paper I mentioned has this enjoyably pragmatic tone: "none of DuckDB's components is revolutionary in its own regard. Instead, we combined methods and algorithms from the state of the art that were best suited for our use cases." This humble position of always being willing to learn and use what's best, and of innovating where necessary really characterizes the project and its community, and I think it also resonates with the engineers that build our data and analytics services, and our customers who build on AWS. Systems evolve as the physics continue to shift, and the embedded engine isn't the end of that story so much as it is a powerful approach that helps carry us forward into new territory. I'm looking forward to seeing where this team takes us.